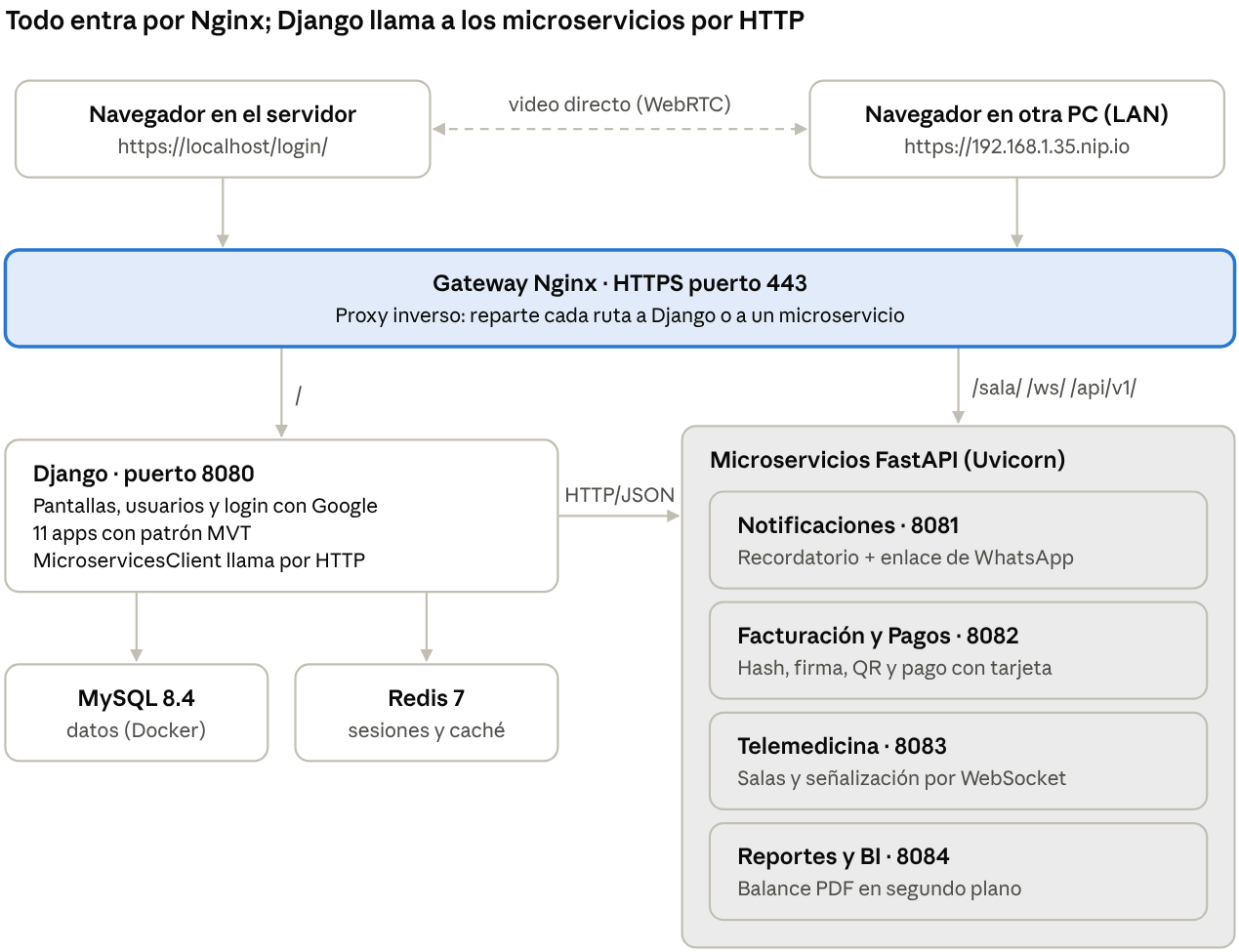
Guía de defensa — VetSystem

Oct 27, 2026 · @Fernando

VetSystem es un sistema veterinario hecho con Django (el núcleo) y 4 microservicios FastAPI independientes, publicados en la red local a través de un gateway Nginx con HTTPS.

Arquitectura de un vistazo



arquitectura de VetSystem · gateway, núcleo Django, 4 microservicios, datos

El navegador nunca habla directo con Django ni con los microservicios: todo pasa por Nginx. Django es el único que toca la base de datos, y llama a los microservicios cuando necesita firmar, notificar o generar un reporte.

Cómo funciona Django

Django es el núcleo del sistema: maneja usuarios, base de datos y todas las pantallas, y sigue el patrón **MVT (Modelo – Vista – Template)**.

- **Modelo** (`models.py`): una clase Python = una tabla de MySQL. Ej.: `Cliente` , `Mascota` , `Cita` , `Factura` . Django crea las tablas con **migraciones** (`manage.py migrate`), sin escribir SQL.
- **Vista** (`views.py`): una función que recibe la petición, consulta los modelos, llama a un microservicio si hace falta y decide qué responder.
- **Template** (`templates/*.html`): el HTML con datos insertados (`{{ factura.total }}`). Todos heredan de `base.html` (menú, mensajes, estilos Bootstrap).
- **URLs** (`urls.py`): asocian cada dirección con su vista. Ej.: `/facturacion/nueva/` → `factura_crear` .

El proyecto está dividido en **11 apps**, cada una un módulo con sus propios modelos, vistas y URLs:

App	Qué maneja
core	Dashboard, login, panel de microservicios, cliente HTTP hacia los microservicios
clientes	Dueños, mascotas y el portal del cliente
agenda	Citas (con opción de teleconsulta)
medico	Historia clínica y vacunas
telemedicina	Videoconsultas ligadas a citas
facturacion	Facturas, detalle, caja diaria
inventario	Productos, stock, proveedores
grooming	Peluquería
reportes	Reportes PDF
usuarios	Personal (veterinarios, recepción)
configuracion	Moneda, impuesto, datos de la clínica

Recorrido de una petición (ej. emitir una factura):

1. El navegador envía el formulario a `https://…/facturacion/nueva/`.
2. Nginx lo recibe y lo pasa a Django (puerto 8080).
3. Los **middleware** revisan la sesión, el token CSRF y si el usuario es cliente o personal.
4. La URL lleva a la vista `factura_crear`, que guarda la factura en MySQL dentro de una transacción.
5. La vista llama al microservicio de Facturación (8082) para firmarla y, si es con tarjeta, a la pasarela.
6. Guarda el hash, firma y QR en la factura y redirige al detalle, que se muestra con su template.

Seguridad que Django da de fábrica: contraseñas con hash, protección CSRF en formularios, `@login_required` en las vistas y sesión que se cierra al cerrar el navegador.

Cómo se conectan los microservicios

Django y los microservicios son **programas separados** que se hablan por **HTTP con JSON (API REST)**, igual que una página web habla con un servidor.

- Cada microservicio es una aplicación **FastAPI** que corre en su propio proceso y puerto: 8081, 8082, 8083 y 8084. Los arranca `microservicios/orchestrator.py` con **Uvicorn** (servidor ASGI).
- Django nunca llama a su código directamente: usa la clase `MicroservicesClient` (`apps/core/services_client.py`), que hace peticiones con la librería `requests`. Ej.: `POST http://127.0.0.1:8082/api/v1/facturacion/emitir-comprobante` con los datos de la factura en JSON.
- El microservicio responde JSON (ej. `hash_cpe`, `firma_digital`, `codigo_qr_texto`) y Django guarda esos datos en MySQL.
- **Cada uno tiene `/health`**: el panel *Monitor de Microservicios* lo consulta para mostrar ONLINE/OFFLINE y la latencia.
- **Cada uno tiene `/docs`** (Swagger): documentación automática de FastAPI donde se pueden probar los endpoints.

Tolerancia a fallos (el punto clave de la arquitectura): todas las llamadas tienen un *timeout* de 1,5–2 segundos y están dentro de `try/except`. Si un microservicio está apagado, Django **no se cae**: la factura o la cita se guardan igual y se muestra un aviso. Eso es *bajo acoplamiento*: cada servicio se puede reiniciar, actualizar o escalar sin tocar el resto.

Por qué microservicios y no todo en Django: las tareas pesadas o externas (firmar comprobantes, generar PDF, video en tiempo real) quedan aisladas; un fallo en una no afecta al núcleo y cada una podría correr en otro servidor solo cambiando su URL (variables `MS_FACTURACION_URL`, etc.).

Limitación a admitir: los microservicios guardan su historial **en memoria**, así que se borra al reiniciarlos. Los datos importantes (hash, firma, QR, autorización) sí quedan guardados en MySQL por Django.

Qué hace cada microservicio

Los cuatro **simulan** servicios externos reales (certificador fiscal, pasarela de pago, WhatsApp, motor de reportes) con el mismo flujo que tendrían en producción.

Microservicio	Puerto	Endpoints principales	Qué hace	Qué se simula
Notificaciones	8081	`POST /api/v1/notificaciones/cita`, `/vacuna`, `GET /historial`	Arma el recordatorio con el teléfono del cliente (le agrega 502 a números de 8 dígitos) y genera un enlace `wa.me` con el mensaje escrito	El envío automático: el personal presiona Enviar en WhatsApp. Lo automático requiere la API de pago de WhatsApp Business
Facturación y Pagos	8082	`POST /api/v1/facturacion/emitir-comprobante`, `POST /api/v1/pagos/procesar-tarjeta`, `GET /comprobantes`	Calcula un hash SHA-256 de la factura, una firma digital y el texto del QR (NIT, serie, IVA, total, DPI, hash). Con tarjeta devuelve un código de autorización	La certificación ante la SAT (FEL) y el cobro real con tarjeta
Telemedicina	8083	`POST /api/v1/telemedicina/crear-sala`, `WS /ws/sala/{id}`, `GET /sala/{id}`	Crea la sala de la cita y hace de **servidor de señalización** por WebSocket; el video y audio viajan directo entre navegadores con **WebRTC**	Nada: la videollamada es real

Microservicio	Puerto	Endpoints principales	Qué hace	Qué se simula
Reportes y BI	8084	`POST /api/v1/reportes/balance-pdf`, `GET /descargar/{archivo}`	Genera el balance financiero en PDF con ReportLab **en segundo plano** (`BackgroundTasks`); Django espera a que esté listo y lo entrega al usuario	Nada: el PDF es real

Cómo funciona la videollamada (WebRTC): los dos navegadores se conectan al WebSocket del microservicio solo para intercambiarse la información de conexión (*offer*, *answer* y candidatos ICE). Después el video va **directo de una PC a la otra** (peer-to-peer), sin pasar por el servidor. El navegador solo permite usar la cámara en un sitio HTTPS: por eso el sistema usa certificado.

Acceso desde otra PC en la misma red

La PC servidor (IP **192.168.1.35**) recibe todo por el puerto 443 (HTTPS) mediante **Nginx**, y las demás PCs de la red entran con `https://192.168.1.35.nip.io/login/`.

Piezas que lo hacen posible:

1. **Escuchar en toda la red, no solo en la PC.** Django arranca con `runserver 0.0.0.0:8080` y los microservicios en `0.0.0.0`. `0.0.0.0` significa "aceptar conexiones de cualquier interfaz", no solo de `127.0.0.1` (localhost).
2. **Gateway Nginx** (contenedor Docker `vetsystem_gateway`, archivo `nginx/lan.conf`). Es un **proxy inverso**: es la única puerta de entrada y reparte cada petición según la ruta:
   - `/` → Django (8080)
   - `/sala/` y `/ws/` → Telemedicina (8083), con soporte de WebSocket (`Upgrade`)
   - `/api/v1/notificaciones/`, `/facturacion/`, `/reportes/` → 8081, 8082, 8084
   - Llega a la PC con `host.docker.internal`, y el puerto 80 redirige a 443.
3. **HTTPS con certificado propio** (`nginx/certs/vetsystem.crt`). Es obligatorio porque el navegador no deja usar la cámara sin HTTPS. Como es autofirmado, la primera vez el navegador avisa: *Avanzado → Continuar*.
4. **Firewall de Windows**: la regla "VetSystem HTTPS" abre los puertos 80 y 443 para que entren conexiones de otras PCs (`abrir_firewall.bat`).

5. **Django confía en el proxy**: `SECURE_PROXY_SSL_HEADER` le dice que la conexión original era HTTPS, y `CSRF_TRUSTED_ORIGINS` acepta formularios que vienen por esas direcciones.
6. **nip.io**: un DNS público gratuito que convierte `192.168.1.35.nip.io` en `192.168.1.35`. Hace falta porque Google no acepta una IP como dirección de retorno del login, pero sí un nombre de dominio.

El lanzador detecta la IP de la red (el adaptador con puerta de enlace) y la muestra al arrancar. Sin internet se puede entrar con `https://192.168.1.35/login/`, pero sin login de Google.

Para que el login con Google funcione desde otra PC, la dirección `https://192.168.1.35.nip.io/accounts/google/login/callback/` debe estar registrada en Google Cloud Console. Si la IP cambia (otra red Wi-Fi), hay que registrar la nueva.

Login con Google y portal del cliente

El login con Google usa **OAuth 2.0** con la librería **django-allauth**: el sistema nunca ve la contraseña de Google, solo recibe el nombre y el correo verificado.

1. El usuario presiona *Continuar con Google* y va a `accounts.google.com`.
2. Elige su cuenta; Google lo devuelve a `/accounts/google/login/callback/` con un código.
3. Django cambia ese código por los datos del usuario (con el *client id* y *secret*, guardados en `local_settings.py`, fuera de Git) e inicia la sesión.

Personal vs. cliente: quien es *staff* (veterinarios, administradores) entra al panel clínico. Una cuenta de Google nueva sin permisos se registra como **cliente**: se crea o vincula su ficha `Cliente` (campo `usuario`) y el `PortalClienteMiddleware` lo encierra en `/portal/`, así nunca ve datos de otros clientes.

Flujo completo para la demo:

1. El cliente entra con Google y completa su perfil (DPI, teléfono, mascota).
2. El administrador lo encuentra en *Clientes* (aparece con la etiqueta **Portal**).
3. Desde su perfil crea una cita marcando **teleconsulta**: en un solo paso se crea la cita y la sala de video.
4. El cliente ve la cita en su portal y entra a la videollamada; el veterinario entra desde *Telemedicina*.

El login con usuario y contraseña sigue disponible para el personal. La sesión expira al cerrar el navegador (o a las 8 horas), por eso siempre pide login al iniciar.

Base de datos, caché y arranque

Los datos viven en **MySQL 8.4** dentro del contenedor Docker `vetsystem_mysql` (puerto 3306); **Redis 7** (`vetsystem_redis`) guarda en memoria las sesiones y la caché del dashboard.

- Django se conecta a MySQL con el ORM: en el código se escribe `Factura.objects.filter(...)` y Django genera el SQL.
- Los cambios de estructura se hacen con migraciones versionadas (ej. `facturacion/0002_comprobante_electronico` agregó hash, firma y QR).
- Si Redis no está, Django usa una caché en memoria local: el sistema sigue funcionando.
- La configuración (base de datos, claves) se lee de variables de entorno, con valores por defecto para desarrollo.

`iniciar_sistema_completo.bat` arranca todo en 6 pasos:

1. Cierra instancias anteriores que ocupen los puertos.
2. Verifica MySQL (reinicia el contenedor si no responde) y levanta Redis.
3. Aplica migraciones pendientes y limpia sesiones viejas (por eso siempre pide login).
4. Inicia los 4 microservicios (8081–8084) en su propia ventana.
5. Inicia el gateway Nginx con HTTPS (puertos 80 y 443).
6. Abre Brave en `https://localhost/login/` , muestra la dirección para otras PCs e inicia Django en el puerto 8080.

Tecnologías: Python 3.13, Django 6.1, FastAPI + Uvicorn, MySQL 8.4, Redis 7, Nginx, Docker, django-allauth, ReportLab, WebRTC, Bootstrap 5.

Preguntas probables del jurado

Pregunta	Respuesta corta
¿Es un monolito o microservicios?	Arquitectura **híbrida**: un núcleo Django modular (11 apps) más 4 microservicios satélite independientes que se comunican por API REST.

Pregunta	Respuesta corta
¿Qué pasa si se cae un microservicio?	Django sigue funcionando: las llamadas tienen timeout y `try/except`. La factura o cita se guarda igual y se muestra un aviso. El panel lo marca OFFLINE.
¿Por qué FastAPI para los microservicios?	Es liviano, asíncrono (ideal para WebSocket y tareas en segundo plano) y genera documentación Swagger automática en `/docs`.
¿Cómo se comunican?	HTTP con JSON. Django usa `requests` desde `MicroservicesClient`; el navegador llega a telemedicina por WebSocket a través de Nginx.
¿Para qué sirve Nginx?	Es el proxy inverso y única puerta de entrada: pone el HTTPS, reparte las rutas a Django o al microservicio correcto y soporta WebSocket.
¿Por qué HTTPS si es una red local?	Los navegadores solo permiten cámara y micrófono en sitios seguros; además cifra contraseñas y sesiones en la red.
¿Cómo entra otra PC?	Por la IP del servidor en la red (`192.168.1.35.nip.io`), puerto 443 abierto en el firewall, Django y los microservicios escuchando en `0.0.0.0`.
¿Qué es nip.io?	Un DNS gratuito que traduce `IP.nip.io` a esa IP; se usa porque Google no acepta IPs como dirección de retorno del login.
¿La videollamada pasa por el servidor?	No. El servidor solo hace la señalización por WebSocket; el video viaja directo entre navegadores con WebRTC (peer-to-peer).
¿La factura es válida ante la SAT?	Es una **simulación** del flujo FEL: hash SHA-256, firma, estado y QR. Para producción se cambiaría el microservicio por la API de un certificador autorizado, sin tocar Django.
¿El WhatsApp se envía solo?	Se genera el mensaje y el enlace `wa.me`; el personal lo envía con un clic. El envío automático requiere la API de pago de WhatsApp Business (Meta).

Pregunta	Respuesta corta
¿Cómo se protege la información?	Login obligatorio, contraseñas con hash, CSRF, sesiones que expiran, clientes aislados en su portal, credenciales de Google fuera del repositorio.
¿Cómo escalaría?	Cada microservicio puede ir a otro servidor cambiando su URL por variable de entorno; con `docker-compose.yml` todo corre en contenedores.
¿Por qué los reportes van en segundo plano?	Generar PDFs es pesado; con `BackgroundTasks` el servicio responde de inmediato y la web no se bloquea.

Para la demo, en este orden: Monitor de Microservicios (4/4 ONLINE) → crear una cita con teleconsulta y mostrar el botón de WhatsApp → videollamada con la otra PC → factura con tarjeta (QR, hash, autorización) → Balance PDF → volver al panel y mostrar el historial.